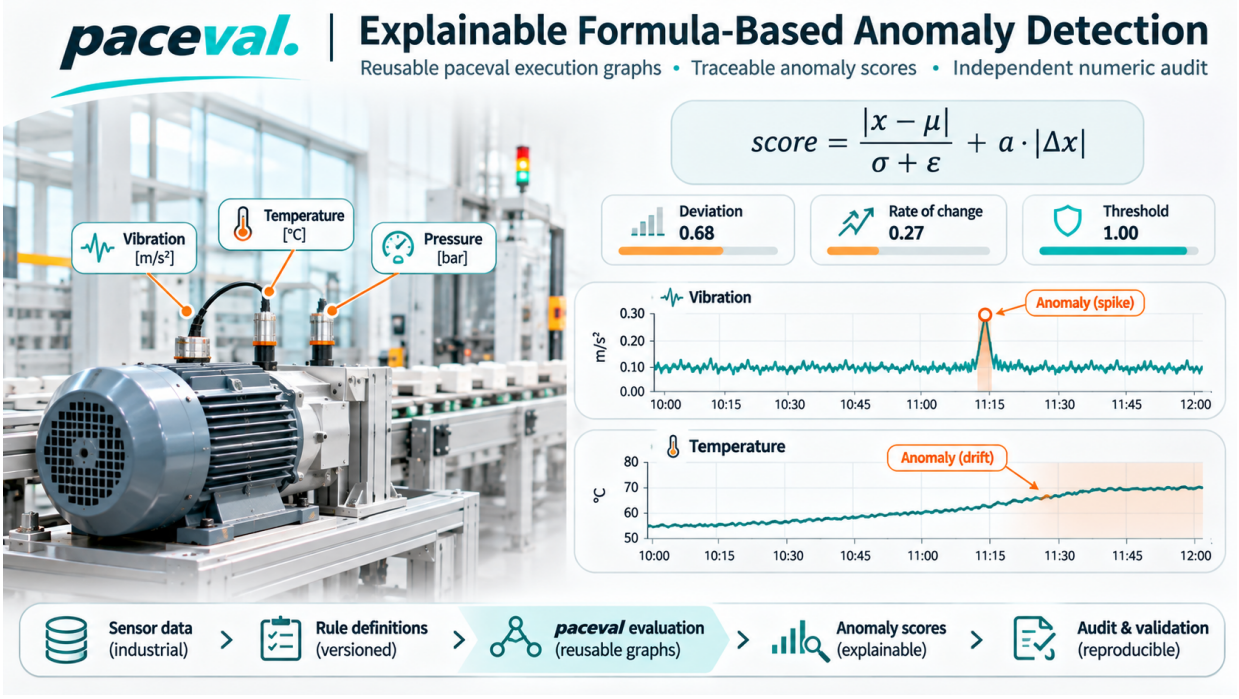
Explainable Formula-Based Anomaly Detection

Versioned industrial sensor scoring with reusable paceval computation graphs, interpretable contributions and independently audited results.



Cover illustration: industrial sensor monitoring and an example anomaly formula. Decorative data on the cover are not research measurements.

Experimental result at a glance

Quantity	Verified observation
Mathematical workload	100 individual score graphs × 4,080 valid sensor samples = 408,000 results
Independent verification	Native C++ reference on all scores; 110-digit independent audit on 200 pairs and 4,080 selected scores
Event outcomes	8/8 held-out synthetic fault events detected (persistence-based alarm path)
Interval diagnostic	24/24 raw TINC samples enclose the exact dyadic reference for the separately tested smooth polynomial

Scientific scope: controlled synthetic, labeled signals; mathematical correctness and traceability are validated. This work does not establish real-world detector effectiveness, realtime operation or safety certification.

1. Research question and executable architecture

WP-017 originally proposed a reusable arithmetic kernel for industrial anomaly scoring. This paper reports a direct experimental implementation, preserving the separation between numerical graph execution, signal metadata, stateful alarm logic and statistical assessment.

The official paceval SDK sources are compiled into the executable and a separately supplied original x86-64 Linux runtime is loaded at execution. Every score uses a paceval computation object. C++ supplies strict data-quality checks, stable model IDs, batched input preparation and a deterministic offline monitoring policy.

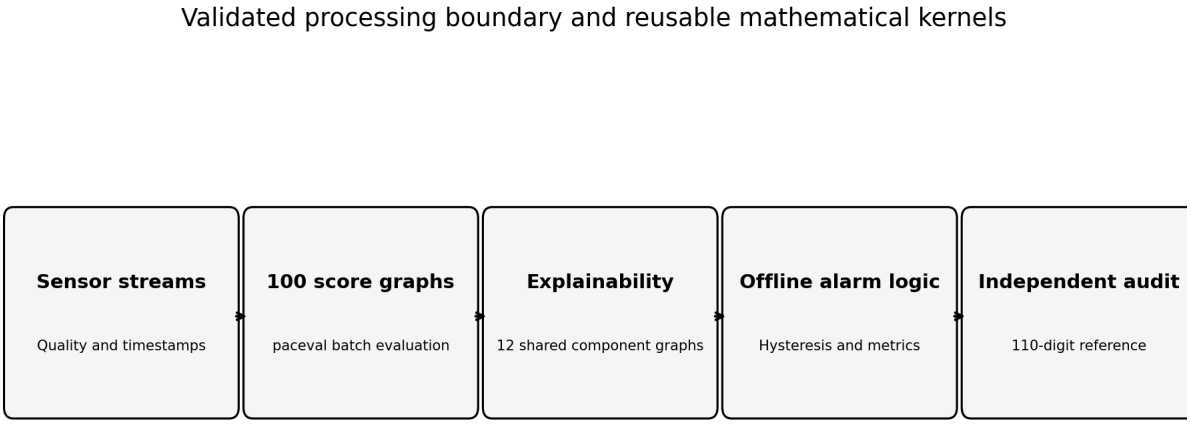


Figure 1. Strict input-quality gating precedes paceval; interpretation, alarm state and audit remain distinct host responsibilities.

The mathematical kernel

Define $z=(x-\mu)/(\sigma+\varepsilon)$, $d=\Delta x/(dscale+\varepsilon)$, and $cn=c/(cscale+\varepsilon)$. These are dimensionless normalization ratios. For each versioned rule, the anomaly score is the sum of three nonnegative terms: a deviation contribution, a change contribution, and a context contribution.

Family	Deviation contribution	Change contribution	Context contribution
0	$w0 \times \lvert z\rvert$	$w1 \times \lvert d\rvert$	$w2 \times \lvert cn\rvert$
1	$w0 \times \lvert z\rvert$	$w1 \times d^2$	$w2 \times \lvert cn\rvert$
2	$w0 \times z^2$	$w1 \times \lvert d\rvert$	$w2 \times \lvert cn\rvert$
3	$w0 \times \lvert z\rvert$	$w1 \times \lvert d\rvert$	$w2 \times cn^2$

The 100 frozen rules each encode their own coefficients and threshold, while 12 shared paceval graphs separately reproduce the three score components for the selected rules. A new rule version can be compiled without modifying the existing 99 rule definitions.

2. Data generation, labels and quality control

Sixteen independent synthetic channels are divided into four instrument classes (temperature, vibration, pressure and electrical monitoring). Four channels represent each class. Channels are split by fixed identity into two training, one validation and one held-out test stream per class. No customer or real plant measurements are present.

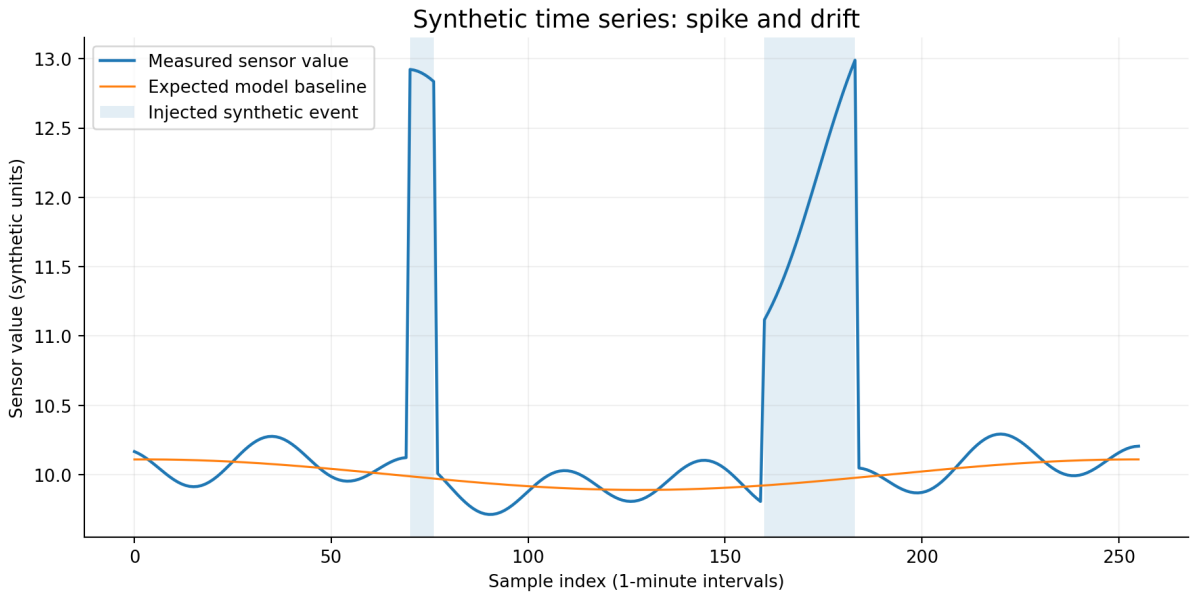


Figure 2. Known injected events: a 7-sample spike and a 24-sample progressive drift in a deterministic 256-minute series.

Every channel has 256 one-minute samples. Known anomalous windows are t=70–76 and t=160–183, determined before scoring. Native labels come from the generator; paceval has no access to labels and does not itself perform statistical classification.

Handling incomplete industrial inputs

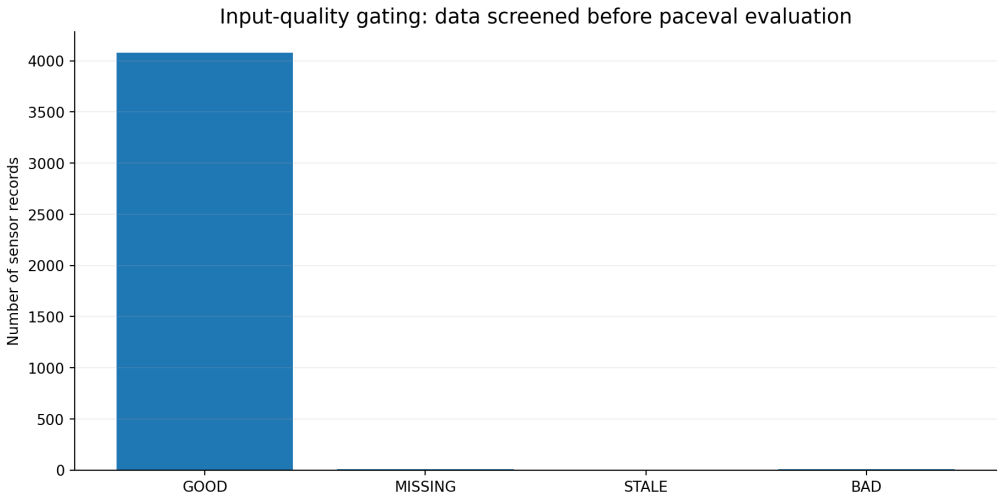


Figure 3. Sixteen missing, stale or bad-quality records are excluded before paceval evaluation.

The strict CSV reader also rejects wrong field counts, non-finite values, invalid normalizations, duplicate timestamps and modifications of the frozen train/validation/test split. The design follows the OPC UA principle of evaluating source timestamps and measurement quality before using measured values.

3. Explainability and controlled formula revisions

A mathematical score is not described as an unexplainable black box. The published selected_scores.csv file records the deviation, change and context contributions produced by paceval, along with the total score, exact rule ID, rule threshold, truth label and alarm state. A strict sum check prevents contributions from silently drifting away from the actual scoring computation.

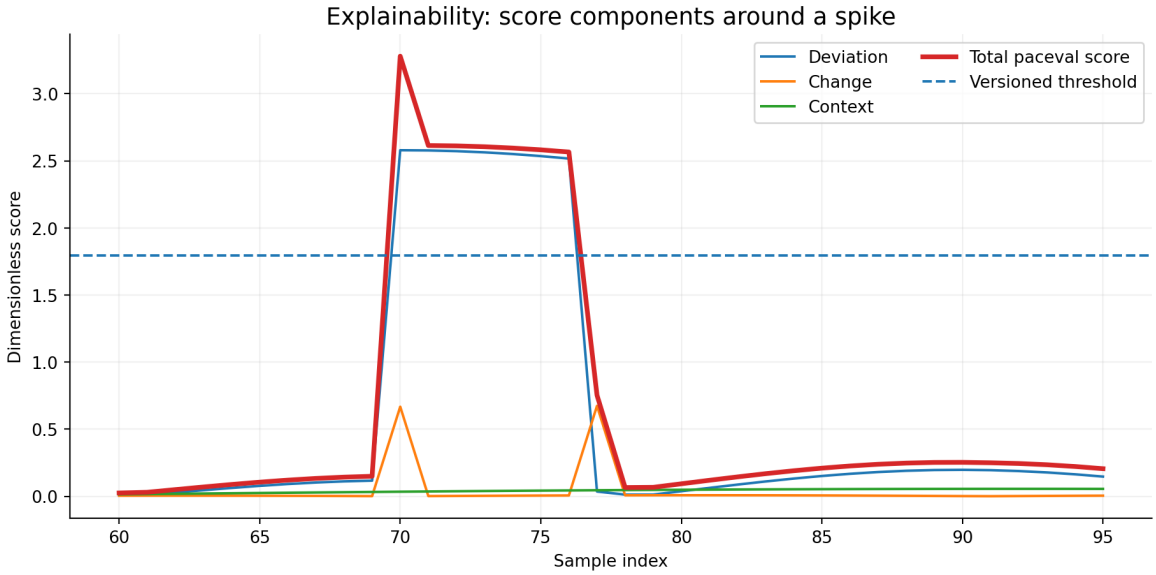


Figure 4. Decomposition of a selected score around a known synthetic spike; all displayed components are evaluated by reusable paceval graphs.

Revision handling is intentionally controlled: rule 17 is recompiled with an updated deviation coefficient; its old result is still auditable. The 99 other score handles remain available and unchanged. Revision does not imply that the altered rule improves anomaly detection or was chosen for deployment.

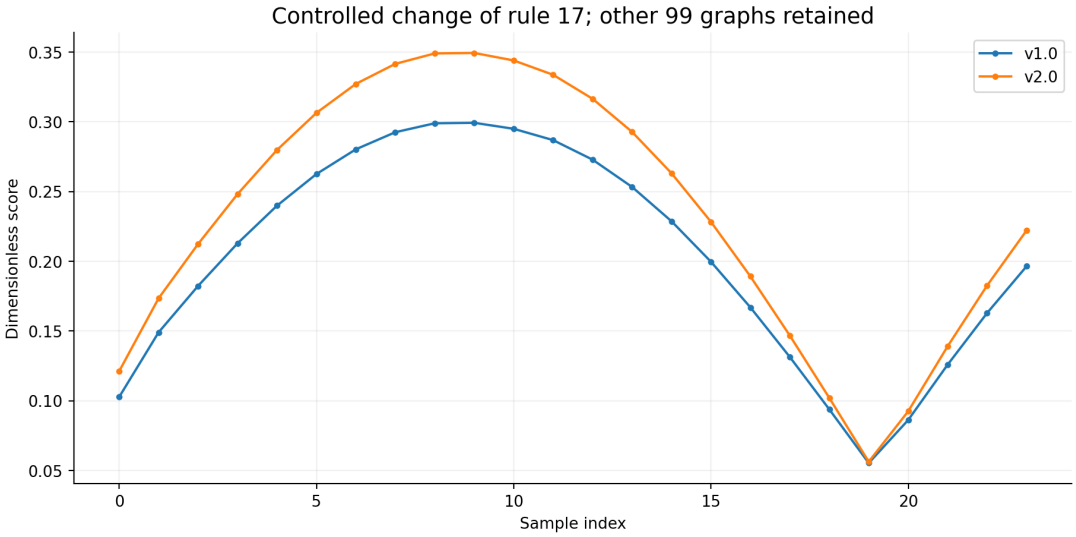


Figure 5. Change comparison of original rule 17 and its deliberately modified second version.

4. Frozen candidate selection and held-out evaluation

Every family has 25 independently versioned formula candidates. Only labeled validation streams determine the winning rule in each class; the tie-breaker favors fewer false positives. The withheld test streams are then reported without further tuning. C++ verifies the expected F1 computations independently of the paceval arithmetic.

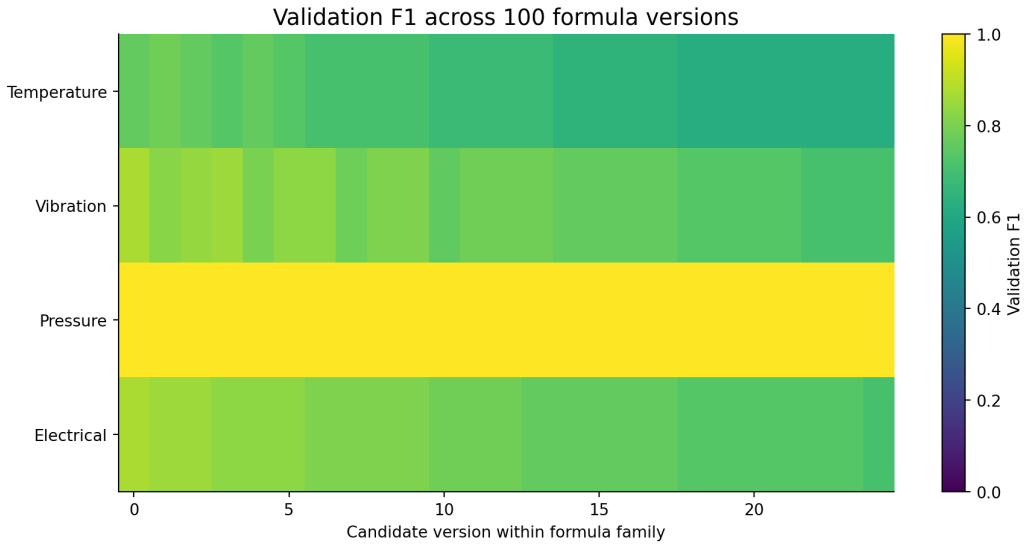


Figure 6. Variation in validation F1 across 100 predeclared formula versions.



Figure 7. Validation versus independently held-out pointwise F1 for the four selected model families.

Raw threshold classification on the selected rules produced 100 true positives, 0 false positives and 24 false negatives on the held-out sample-level labels. These counts do not include the additional stateful-alarm persistence behavior. Because synthetic anomaly patterns are known and simple, these numbers are not general predictive-performance estimates.

5. Alarm persistence, event recall and detection delay

A positive alarm state starts after two successive above-threshold scores and clears after two successive below-70%-threshold readings. Bad-quality input resets the alarm state. The evaluation explicitly distinguishes pointwise rule scores, positive alarm states, and event-level detection. Persistence can extend an alarm after the end of an event.

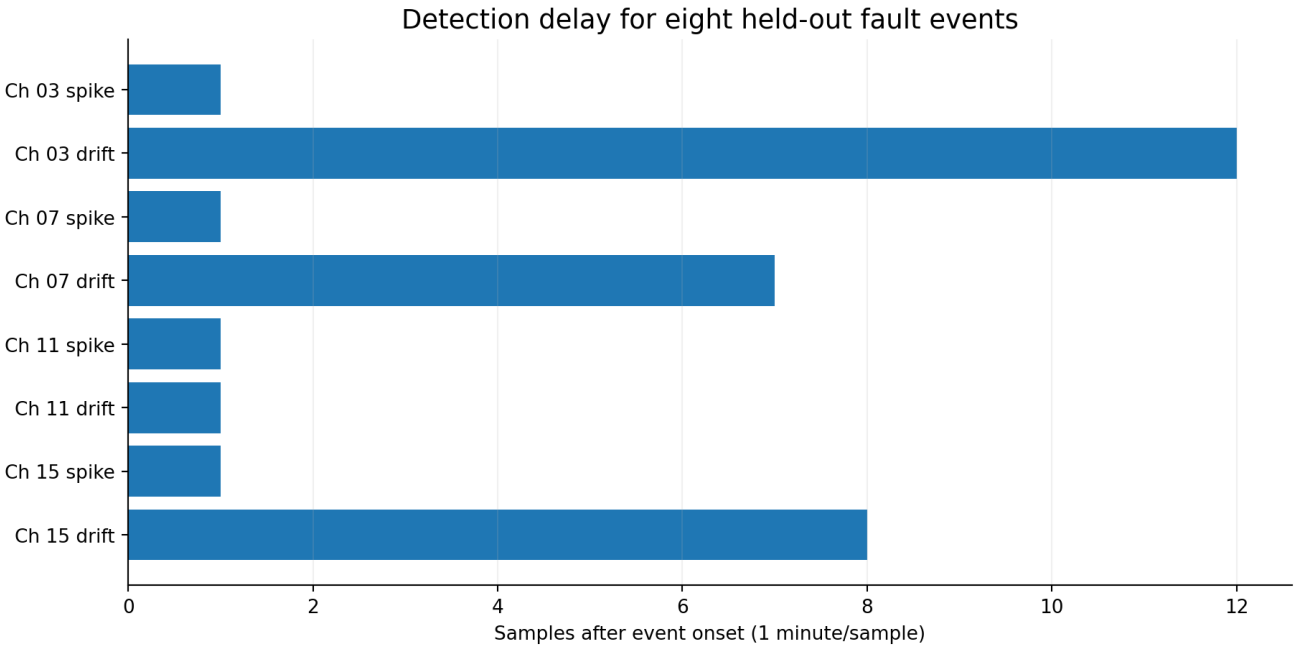


Figure 8. First detection time after each of the eight independently held-out synthetic event onsets.

Outcome	Held-out synthetic result
Injected fault events	8 = 4 channels × (1 spike + 1 drift)
Detected fault events	8/8
Detection-delay range	1–12 one-minute samples
Raw threshold false positives	0 on the held-out samples
Persistent state false-positive samples	10 on held-out samples (post-event persistence)

The event-level observation does not establish performance at other anomaly prevalence levels, operational regimes or real physical failure mechanisms. It is nonetheless a reproducible way to distinguish mathematics, stateful policy and detection assessment.

6. Independent numerical validation and interval observation

The original paceval runtime performed 408,000 score evaluations. The largest native C++ absolute score difference was 7.105e-15. A separate 110-digit Decimal reference, reconstructing exact binary64 inputs with Decimal.from_float, verified 200 selected score/rule pairs and all 4,080 reported selected scores and their explanation components. The maximum high-precision difference for the selected 200 score samples was approximately 1.22×10^{-15}.

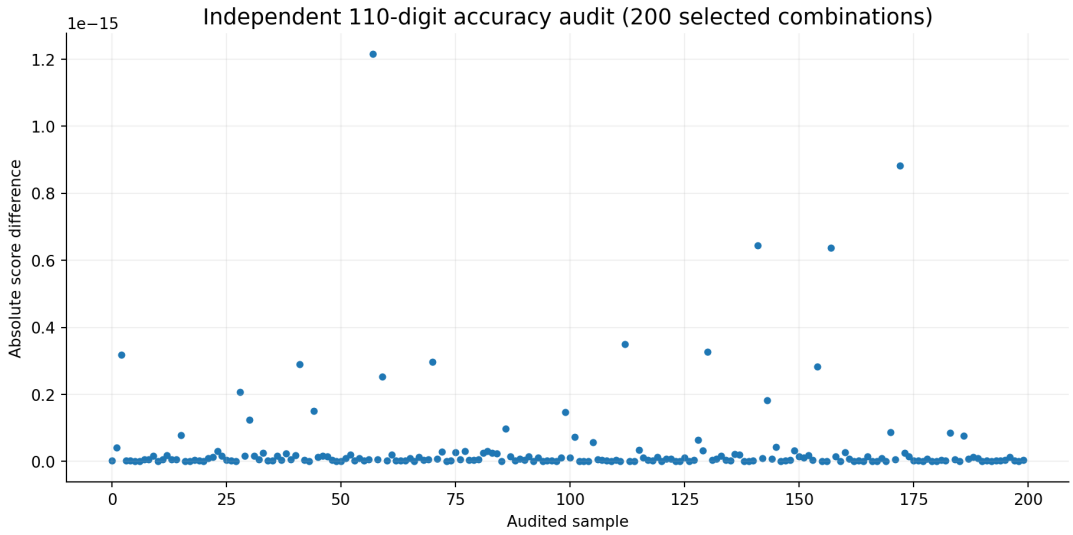


Figure 9. Absolute residuals for 200 independently chosen, high-precision-recomputed rule/sample combinations.

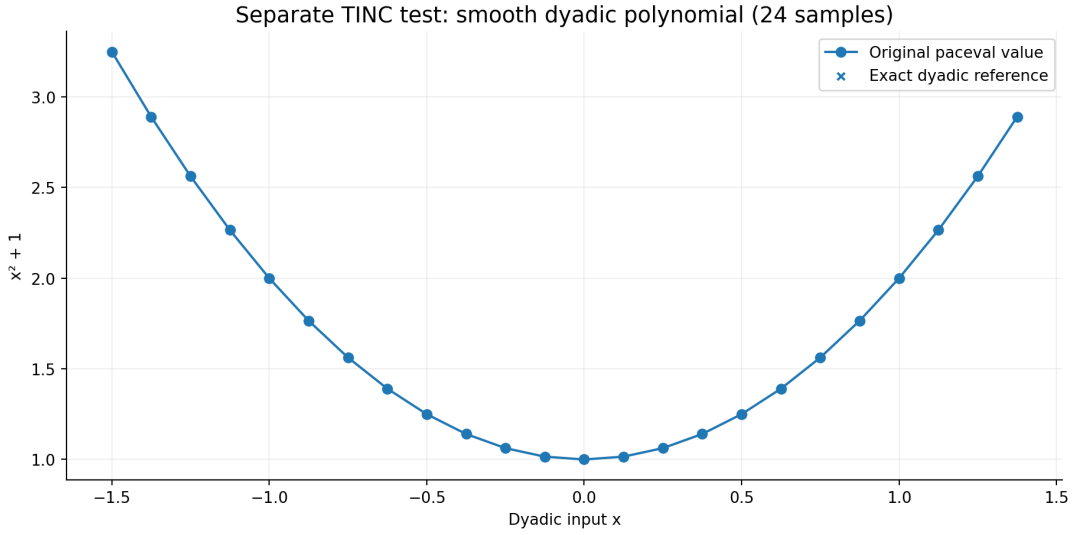


Figure 10. Separate 24-sample interval diagnostic using $x^2 + 1$ on exactly representable dyadic inputs.

All 24 original TINC intervals contain their separate exact reference value for this smooth dyadic control expression. This observation is not a formal proof of outward rounding or a global enclosure claim for nonsmooth scores containing $|\cdot|$. No post-hoc widening of the raw reported intervals was applied.

7. Reproducible implementation and tested environment

The executable is written in C++17 with CMake. The two original paceval SDK sources live in src/, not in a separate library folder. The externally supplied shared library remains an explicit deployment dependency. The official source notices are retained. No extra license-notice file is created.

Actual SDK function	Purpose in this demonstrator
paceval_InitializeLibrary	Initialize original Linux runtime
paceval_CreateComputation	Build 100 versioned score graphs, 12 explanation graphs, one TINC graph
paceval_dGetMultipleComputationsResultsExt	Evaluate many computation handles over a batch of sensor parameter rows
paceval_dGetComputationResult	Evaluate changed rule and independent TINC diagnostic
paceval_GetIsError / paceval_GetErrorInformation	Check computation/compile failures
paceval_DeleteComputation / paceval_FreeLibrary	Release original objects and runtime

Tested setup

Debian GNU/Linux 13 (trixie), x86-64, AMD EPYC 9V74 virtualized processor, 5 logical CPUs visible in this environment, GCC 14.2 and Clang 17. Original separately supplied paceval Linux shared object. AddressSanitizer and UndefinedBehaviorSanitizer were also exercised. These environment details describe the validation platform and are not throughput claims.

Reproduction commands

1. cmake -S src -B build -DPACEVAL_RUNTIME=/absolute/path/to/original_paceval.so
2. cmake --build build --parallel; ctest --test-dir build --output-on-failure
3. ./build/anomaly_detection --out regenerated --batch 37
4. python3 src/validate_outputs.py --examples regenerated
5. python3 src/make_figures.py

The source tests include a byte-identical CSV replay, explicit malformed-input cases and an independent high-precision reference. See README.md and examples/validation.json for all exact paths.

8. Scientific interpretation and future research

This is a finished, reproducible synthetic feasibility study rather than a deployment claim. It establishes that a family of versioned anomaly formulas can be executed by persistent paceval computation objects and that their component attribution is independently auditable. The host rather than paceval owns data validity, anomaly-label construction, model-selection discipline, hysteresis, event scoring, and safety-related decisions.

What remains to be validated

The Numenta Anomaly Benchmark highlights detection delay and false-alarm costs in time-series monitoring. A meaningful deployment study should therefore evaluate archived, independently labeled industrial fault streams and measure event detection under varying regimes, sensor drift, class prevalence and operationally specified false-alarm costs. The current synthetic thresholds and reference baselines have not been shown to transfer to a physical plant.

Operational extensions include native OPC UA ingestion with verified source timestamps and quality bits, external validated rule registries, real streaming p99 latency measurements, forensic audit-log retention and model governance. A kernel timing observation alone would not establish a valid anomaly detector.

Selected external references

1. paceval API, C/C++ execution objects and batch evaluation: https://paceval.com/api/

2. Numenta Anomaly Benchmark, v1.1, open benchmark corpus and scoring protocol: https://github.com/numenta/NAB

3. Ahmad et al., Evaluating Real-Time Anomaly Detection Algorithms, ICMLA 2015: https://doi.org/10.1109/ICMLA.2015.141

4. OPC Foundation, OPC UA Part 4, DataValue and SourceTimestamp specification: https://reference.opcfoundation.org/specs/OPC-10000-4/7.11

5. paceval C/C++ sources and release information: https://github.com/paceval/paceval

Publication artifacts

The whitepaper is distributed together with README.md, src/anomaly_detection.cpp, src/paceval_main.h, src/paceval_main.cpp, src/validate_outputs.py, examples/selected_scores.csv, examples/event_detection.csv, examples/validation.json, examples/independent_audit.json and figures/01–10. All data in this demonstration are synthetic and generated deterministically by the published source code.

Conclusion: a reusable and explainable anomaly-score mathematical execution layer was successfully demonstrated with paceval and independently validated on a defined offline workload. The actual detection outcomes should be interpreted only within that workload.